



Spring 2026

Department of Computer Science

Bahria University, Karachi Campus

Name: Fatima Ansar

Department: BS-CS

Semester: (3)

Course: Data Structure and Algorithm

Table of Content

Sr. No.	Contents	Page No.
1	Title Page	1
2	Table of Contents	2
3	Problem Statement	3
4	Code for Task 1	5
5	Outputs of Task 1	12
6	Code for Task 2	16
7	Outputs of Task 2	23
8	Time Complexity Analysis of Task 1	27
9	Time complexity Analysis of Task 2	37
10	Food Distribution Network Model	48
11	Comparison of Networks	58
12	Explanation of Data Structures	59
13	Justification for Task 1 Data Structures	60
14	Justification for Task 2 Data Structures	61
15	Justification for Task 3 Data Structures(Distribution Model Graph)	64



Spring 2026
 Department of Computer Science
 Bahria University, Karachi Campus

Problem

Poverty alleviation is the most persistent challenge facing Pakistan since its inception. The underlying factors that create and perpetuate poverty involve a number of dimensions, e.g. income inequality, unemployment or underemployment, inadequate access to basic services and resources by the poor; inequitable distribution of assets, technology and socioeconomic opportunities; and underdeveloped infrastructure, etc.

The increase of oil prices due to ongoing war also impacts poverty due to increase the prices of fuel as well as the price for the goods.

Consider students third semester of computer science of BUKC are working in Karachi on their **CSP**(Community Support Program) activity that is mandatory part of their degree program. They decide to tackle hunger in underprivileged areas of Karachi by building a *food redistribution app(based on demand and supply)*. This App is a system that connects surplus food suppliers (restaurants, grocery stores, farms) with underprivileged communities. The system must minimize food waste and ensure timely delivery of perishable items. For designing this app, apply appropriate data structure that build the above system.

Requirements.

1. Model suppliers, warehouses, and communities.
2. Model the areas of underprivileged people. identify the number of people, amount of food required.
3. Provide the food first where poverty is higher.
4. Distribute the food perishable items first.
5. Model the distribution route where food is distributed in cost effective manner where cost of fuel is minimized.

Tasks

1. **Create the code in C++ for Requirement 1 & 2** (CLO4, PLO4, C6)
 - a. Apply appropriate data structures to model suppliers, warehouses, and communities.
 - b. Represent underprivileged areas with attributes such as population size and food requirements.
 - c. Ensure time complexity does not exceed $O(\log n)$.



Spring 2026
Department of Computer Science
Bahria University, Karachi Campus

2. Create the code C++ for Requirement 3& 4 (CLO4, PLO4, C6)

- a. Use efficient data structures for food distribution to high-poverty areas.
- b. Ensure perishable items are delivered first.

3. Cost-Effective Distribution Model for requirement 5 (CLO4, PLO4, C6)

- a. Construct a network of at least 15 distribution areas.
- b. Optimize routes to minimize fuel costs.

Note: Code is not required.

2. Time Complexity Analysis (CLO3, PLO2, C4)

- o Provide detailed analysis of the algorithms used in Task 1 and Task 2.

3. Explanation of Data Structures:

- o Justify the choice of data structures in terms of efficiency, scalability, and suitability for the problem. (CLO1, PLO1, C2)

Deliverables.

1. Code for task 1 and 2 with the output.
2. Time complexity Analysis.
3. Networks for food distribution.
4. Explanation of data structures.
5. Comparison of distribution network with cost effective network.

Note: Single file is required with title page, index page, codes with output, calculations and comparisons.

Evaluation Criteria

Evaluation Criteria & Targeted CLOs

Deliverables	Evaluation Criteria	Targeted CLOs
Code and network designing	50 %	CLO-4,PLO-4,C6
Time complexity Analysis	25 %	CLO-3,PLO-2,C4
Comparative study and use of data structures	25%	CLO-1,PLO-1,C2

CODE FOR TASK 1: (BST)

```

#include<iostream>
#include<string>
using namespace std;

struct supplier {
    string s_name;
    string type;           //restraunt, farm,supermarket
    int food_available;    // surplus(extra) food in kg
};

struct warehouse {
    string w_name;
    string location;
    int capacity;          //max kg it can hold
    int curr_stock;        //current kg it has
};

struct community {
    string area_name;
    int population;        //no of people in that area
    int food_required;     // kg of food required per day
    int poverty_level;     // 1 to 10. (10 = most in need)
};

struct BST_Node {
    community data;
    BST_Node* left;
    BST_Node* right;
};

BST_Node* root = NULL;

BST_Node* create_node(community c) {    //community c = data to be inserted
    BST_Node* temp = new BST_Node();
    temp->data = c;
    temp->left = NULL;
    temp->right = NULL;
    return temp;
}

BST_Node* insert(BST_Node* root, community c) {
    if (root == NULL) {
        BST_Node* root = create_node(c);
        return root;
    }
    if (c.poverty_level < root->data.poverty_level) {
        root->left = insert(root->left, c);
    }
}

```

```

        return root;
    }
    if (c.poverty_level > root->data.poverty_level) {
        root->right = insert(root->right, c);
        return root;
    }
}

BST_Node* search(BST_Node* root, int p_level_to_find) {
    if (root == NULL) {
        cout << "\nNo Area found with poverty level: " << p_level_to_find << "\n\n";
        return NULL;
    }
    if (p_level_to_find == root->data.poverty_level) {
        return root;
    }
    else if (p_level_to_find < root->data.poverty_level) {
        return search(root->left, p_level_to_find);
    }
    else {
        return search(root->right, p_level_to_find);
    }
}
//lowest to highest sorted poverty areas (IN ORDER TRAVERSAL)

```

```

void In_Order(BST_Node* root) {
    if (root != NULL) {
        In_Order(root->left);
        cout << "\nArea: " << root->data.area_name;
        cout << "\nPopulation: " << root->data.population;
        cout << "\nFood Required: " << root->data.food_required << " kg";
        cout << "\nPoverty Level: " << root->data.poverty_level << "/10\n";
        cout << "\n-----\n";
        In_Order(root->right);
    }
}

```

```

void display_suppliers(supplier suppliers[], int count) {
    cout << "\n ----- \n";
    cout << "          TOTAL SUPPLIERS          \n";
    cout << " ----- \n";
    for (int i = 0; i < count; i++) {
        cout << " " << i + 1 << ". " << suppliers[i].s_name
            << " [" << suppliers[i].type << "]"
            << " -- " << suppliers[i].food_available << " kg available\n";
    }
}

```

```

}
void display_warehouses(warehouse warehouses[], int count) {
    cout << "\n ----- \n";
    cout << "          TOTAL WAREHOUSES          \n";
    cout << " ----- \n";
    for (int i = 0; i < count; i++) {
        cout << " " << i + 1 << ". " << warehouses[i].w_name
            << " | Location : " << warehouses[i].location
            << " | Capacity: " << warehouses[i].capacity << " kgs"
            << " | Current stock: " << warehouses[i].curr_stock << " kgs\n";
    }
}

void display_communities(BST_Node* root) {
    cout << "\n ----- \n";
    cout << "    UNDERPRIVILEGED AREAS    \n";           //(BST Sorted)
    cout << "    Order: Lowest to Highest Poverty    \n";
    cout << " ----- \n";
    In_Order(root);
}

void input_supplier(supplier suppliers[], int startIndex, int count) {
    for (int i = startIndex; i < startIndex + count; i++) {
        cout << "\n >>> Supplier " << i + 1 << " <<<\n\n";
        cout << " Enter Name : ";
        cin.ignore();
        getline(cin, suppliers[i].s_name);
        cout << " Enter type (Restaurant / Farm / Supermarket) : ";
        getline(cin, suppliers[i].type);
        cout << " Food Available (kg) : ";
        cin >> suppliers[i].food_available;
    }
}

void input_warehouse(warehouse warehouses[], int startIndex, int count) {
    for (int i = startIndex; i < startIndex + count; i++) {
        cout << "\n >>> Warehouse " << i + 1 << " <<<\n\n";
        cout << " Enter Name : ";
        cin.ignore();
        getline(cin, warehouses[i].w_name);
        cout << " Enter Location : ";
        getline(cin, warehouses[i].location);
        cout << " Enter Capacity (kgs) : ";
        cin >> warehouses[i].capacity;
        cout << " Enter Current Stock (kgs) : ";
        cin >> warehouses[i].curr_stock;
    }
}

```

```
//Taking input for each community and inserting into BST
// communityNumber uses updated number for display
BST_Node* input_community(BST_Node* root, int count, int communityNumber) {
    for (int i = 0; i < count; i++) {
        community c;
        cout << "\n >>> Community " << communityNumber << " <<<\n\n";
        cout << "   Enter Area Name           : ";
        cin.ignore();
        getline(cin, c.area_name);
        cout << "   Enter Population             : ";
        cin >> c.population;
        cout << "   Enter Food Required (kg/day) : ";
        cin >> c.food_required;
        cout << "   Enter Poverty Level (1 to 10) : ";
        cin >> c.poverty_level;

        while (c.poverty_level < 1 || c.poverty_level > 10) {
            cout << "\n   Invalid! Enter between 1 and 10 : ";
            cin >> c.poverty_level;
        }

        root = insert(root, c);
        cout << "   Community added successfully!\n";
        communityNumber++; // increment for next community
    }
    return root;
}

int main() {
    cout << " !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n";
    cout << " - WELCOME TO FOOD REDISTRIBUTION SYSTEM - \n";
    cout << " !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n";

    // HARDCODED SUPPLIERS
    supplier suppliers[100];
    int supplierCount = 3;

    suppliers[0].s_name = "Kababjees Tariq Road";
    suppliers[0].type = "Restaurant";
    suppliers[0].food_available = 500;

    suppliers[1].s_name = "Hay Day Farm";
    suppliers[1].type = "Farm";
    suppliers[1].food_available = 200;

    suppliers[2].s_name = "Imtiaz";
    suppliers[2].type = "Supermarket";
```



```

suppliers[2].food_available = 80;

display_suppliers(suppliers, supplierCount);

// ASK USER TO ADD MORE SUPPLIERS
string choice;
cout << "\nDo you want to add more suppliers? (yes/no) : ";
cin >> choice;

while (choice == "yes" || choice == "Yes") {
    int more;
    cout << "How many suppliers to add? : ";
    cin >> more;
    input_supplier(suppliers, supplierCount, more);
    supplierCount += more;
    display_suppliers(suppliers, supplierCount);
    cout << "\nDo you want to add more suppliers? (yes/no) : ";
    cin >> choice;
}

// HARDCODED WAREHOUSES
warehouse warehouses[100];
int warehouseCount = 3;

warehouses[0].w_name = "Central Hub";
warehouses[0].location = "Saddar";
warehouses[0].capacity = 500;
warehouses[0].curr_stock = 320;

warehouses[1].w_name = "North Storage";
warehouses[1].location = "North Nazimabad";
warehouses[1].capacity = 300;
warehouses[1].curr_stock = 150;

warehouses[2].w_name = "East Depot";
warehouses[2].location = "Gulshan-e-Iqbal";
warehouses[2].capacity = 400;
warehouses[2].curr_stock = 200;

display_warehouses(warehouses, warehouseCount);

// ASK USER TO ADD MORE WAREHOUSES
cout << "\nDo you want to add more warehouses? (yes/no) : ";
cin >> choice;

while (choice == "yes" || choice == "Yes") {

```

```

    int more;
    cout << "How many warehouses to add? : ";
    cin >> more;
    input_warehouse(warehouses, warehouseCount, more);
    warehouseCount += more;
    display_warehouses(warehouses, warehouseCount);
    cout << "\nDo you want to add more warehouses? (yes/no) : ";
    cin >> choice;
}
// HARDCODED COMMUNITIES INTO BST
community c1;
c1.area_name = "Orangi";
c1.population = 800000;
c1.food_required = 2000;
c1.poverty_level = 6;

community c2;
c2.area_name = "Malir";
c2.population = 200000;
c2.food_required = 600;
c2.poverty_level = 5;

community c3;
c3.area_name = "Nazimabad";
c3.population = 150000;
c3.food_required = 400;
c3.poverty_level = 7;

community c4;
c4.area_name = "Lyari";
c4.population = 500000;
c4.food_required = 1500;
c4.poverty_level = 8;

community c5;
c5.area_name = "Korangi";
c5.population = 300000;
c5.food_required = 900;
c5.poverty_level = 4;

root = insert(root, c1);
root = insert(root, c2);
root = insert(root, c3);
root = insert(root, c4);
root = insert(root, c5);

```

```

display_communities(root);

// communityNumber starts at 6 since 5 are already inserted
int communityNumber = 6;

// ASK USER TO ADD MORE COMMUNITIES
cout << "\nDo you want to add more communities? (yes/no) : ";
cin >> choice;

while (choice == "yes" || choice == "Yes") {
    int more;
    cout << "How many communities to add? : ";
    cin >> more;
    root = input_community(root, more, communityNumber);
    communityNumber += more; // update for next round
    display_communities(root);
    cout << "\nDo you want to add more communities? (yes/no) : ";
    cin >> choice;
}

// SEARCH BY POVERTY LEVEL
cin.ignore();
do {
    cout << "\nDo you want to search any community by poverty level? (yes/no) : ";
    getline(cin, choice);

    if (choice == "yes" || choice == "Yes") {
        int level;
        cout << "\nEnter poverty level: ";
        cin >> level;
        BST_Node* result = search(root, level);
        if (result != NULL) {
            cout << "\n Area Found:\n";
            cout << " Name      : " << result->data.area_name << "\n";
            cout << " Population : " << result->data.population << " people\n";
            cout << " Food Needed : " << result->data.food_required << " kg/day\n";
            cout << " Poverty Level: " << result->data.poverty_level << "/10\n";
        }
        cin.ignore();
    }
} while (choice != "no" && choice != "No");

cout << "\n\n-----";
cout << "\nALL DETAILS FETCHED SUCCESSFULLY.\n";
cout << "-----";
return 0; }

```

OUIHfUHS:

- Displaying suppliers and asking user to add more

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
- WELCOME TO FOOD REDISTRIBUTION SYSTEM -
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

-----
TOTAL SUPPLIERS
-----
1. Kababjees Tariq Road [Restaurant] -- 500 kg available
2. Hay Day Farm [Farm] -- 200 kg available
3. Imtiaz [Supermarket] -- 80 kg available

Do you want to add more suppliers? (yes/no) : yes
How many suppliers to add? : 1

>>> Supplier 4 <<<

Enter Name : Chase up
Enter type (Restaurant / Farm / Supermarket) : Supermarket
Food Available (kg) : 400

-----
TOTAL SUPPLIERS
-----
1. Kababjees Tariq Road [Restaurant] -- 500 kg available
2. Hay Day Farm [Farm] -- 200 kg available
3. Imtiaz [Supermarket] -- 80 kg available
4. Chase up [Supermarket] -- 400 kg available

Do you want to add more suppliers? (yes/no) : no

```

- Displaying warehouses and asking user to add more

```

-----
TOTAL WAREHOUSES
-----
1. Central Hub | Location :Saddar | Capacity: 500 kgs | Current stock: 320 kgs
2. North Storage | Location :North Nazimabad | Capacity: 300 kgs | Current stock: 150 kgs
3. East Depot | Location :Gulshan-e-Iqbal | Capacity: 400 kgs | Current stock: 200 kgs

Do you want to add more warehouses? (yes/no) : yes
How many warehouses to add? : 1

>>> Warehouse 4 <<<

Enter Name : saddar
Enter Location : saddar west
Enter Capacity (kgs) : 200
Enter Current Stock (kgs) : 80

-----
TOTAL WAREHOUSES
-----
1. Central Hub | Location :Saddar | Capacity: 500 kgs | Current stock: 320 kgs
2. North Storage | Location :North Nazimabad | Capacity: 300 kgs | Current stock: 150 kgs
3. East Depot | Location :Gulshan-e-Iqbal | Capacity: 400 kgs | Current stock: 200 kgs
4. saddar | Location :saddar west | Capacity: 200 kgs | Current stock: 80 kgs

Do you want to add more warehouses? (yes/no) : no

```

- Displaying underprivileged areas and asking user to add more

```
-----  
UNDERPRIVILEGED AREAS  
Order: Lowest to Highest Poverty  
-----  
  
Area: Korangi  
Population: 300000  
Food Required: 900 kg  
Poverty Level: 4/10  
  
-----  
  
Area: Malir  
Population: 200000  
Food Required: 600 kg  
Poverty Level: 5/10  
  
-----  
  
Area: Orangi  
Population: 800000  
Food Required: 2000 kg  
Poverty Level: 6/10  
  
-----  
  
Area: Nazimabad  
Population: 150000  
Food Required: 400 kg  
Poverty Level: 7/10  
  
-----
```

```
Area: Nazimabad  
Population: 150000  
Food Required: 400 kg  
Poverty Level: 7/10  
  
-----  
  
Area: Lyari  
Population: 500000  
Food Required: 1500 kg  
Poverty Level: 8/10  
  
-----  
  
Do you want to add more communities? (yes/no) : yes  
How many communities to add? : 1  
  
>>> Community 6 <<<  
  
Enter Area Name : saddar  
Enter Population : 220000  
Enter Food Required (kg/day) : 3000  
Enter Poverty Level (1 to 10) : 9  
Community added successfully!
```

- **Re-displaying underprivileged areas and asking user to add more**

```
-----  
UNDERPRIVILEGED AREAS  
Order: Lowest to Highest Poverty  
-----  
  
Area: Korangi  
Population: 300000  
Food Required: 900 kg  
Poverty Level: 4/10  
  
-----  
  
Area: Malir  
Population: 200000  
Food Required: 600 kg  
Poverty Level: 5/10  
  
-----  
  
Area: Orangi  
Population: 800000  
Food Required: 2000 kg  
Poverty Level: 6/10  
  
-----  
  
Area: Nazimabad  
Population: 150000  
Food Required: 400 kg  
Poverty Level: 7/10  
  
-----
```

```
-----  
  
Area: Lyari  
Population: 500000  
Food Required: 1500 kg  
Poverty Level: 8/10  
  
-----  
  
Area: saddar  
Population: 220000  
Food Required: 3000 kg  
Poverty Level: 9/10  
  
-----  
  
Do you want to add more communities? (yes/no) : no
```

- Asking user to search community by entering its poverty level

```
Do you want to search any community by poverty level? (yes/no) : yes
```

```
Enter poverty level: 1
```

```
No Area found with poverty level: 1
```

```
Do you want to search any community by poverty level? (yes/no) : yes
```

```
Enter poverty level: 8
```

```
Area Found:
```

```
Name      : Lyari
```

```
Population : 500000 people
```

```
Food Needed : 1500 kg/day
```

```
Poverty Level: 8/10
```

```
Do you want to search any community by poverty level? (yes/no) : no
```

```
-----  
ALL DETAILS FETCHED SUCCESSFULLY.  
-----
```

CODE FOR TASK 2: (HEAP (Max/Min))

```
#include<iostream>
#include<string>
using namespace std;

struct community {
    string c_name;
    int poverty_level;
};
struct food_item {
    string f_name;
    int daysleft;
};
// MAX HEAP STRUCT AND FUNCTIONS (Req 3)
// Highest poverty score served first

struct Max_heap {
    community arr[100];
    int size;
};

//set max heap downwards from index i

void heapify_max(Max_heap& h, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // if left child exists and is greater than current largest
    if (left < h.size && h.arr[left].poverty_level > h.arr[largest].poverty_level)
        largest = left;

    // if right child exists and is greater than current largest
    if (right < h.size && h.arr[right].poverty_level > h.arr[largest].poverty_level)
        largest = right;

    // if largest is not current index, swap and continue fixing downward
    if (largest != i) {
        swap(h.arr[i], h.arr[largest]);
        heapify_max(h, largest);
    }
}
```



```

void build_max_heap(Max_heap& h) {
    for (int i = (h.size / 2) - 1; i >= 0; i--)
        heapify_max(h, i);
}
// Insert new community into max heap
void insert_area(Max_heap& h, community c) {

    h.arr[h.size] = c;           //new community at the end of array
    h.size++;
    int i = h.size - 1;         //index of new inserted community
    while (i > 0) {
        int parent = (i - 1) / 2; //since our array index starts from 0
        if (h.arr[parent].poverty_level < h.arr[i].poverty_level) {
            swap(h.arr[i], h.arr[parent]);
            i = parent;           //it will keep updating until parent is larger
        }
        else break;
    }
}
// Remove and return highest poverty area (always at index 0)
community removeTopArea(Max_heap& h) {
    community top = h.arr[0];
    h.arr[0] = h.arr[h.size - 1];
    h.size--;
    heapify_max(h, 0);           //heap will keep updating from top till it is reached to its
    //deserving index
    return top;
}
// MIN HEAP STRUCT AND FUNCTIONS (Req 4)
// Soonest expiry(perishable) food distributed first
struct Min_heap {
    food_item arr[100];
    int size;
};
void heapify_min(Min_heap& h, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // if left child exists and is smaller than current smallest

    if (left < h.size && h.arr[left].daysleft < h.arr[smallest].daysleft)
        smallest = left;

    // if right child exists and is smaller than current smallest

```

```

    if (right < h.size && h.arr[right].daysleft < h.arr[smallest].daysleft)
        smallest = right;

    // if smallest is not current index, swap and continue fixing downward

    if (smallest != i) {
        swap(h.arr[smallest], h.arr[i]);
        heapify_min(h, smallest);
    }
}
// Builds min heap from unsorted array all at once

void build_min_heap(Min_heap& h) {
    for (int i = (h.size) / 2 - 1; i >= 0; i--)
        heapify_min(h, i);
}
// Insert new food item into min heap

void insert_food(Min_heap& h, food_item f) {
    h.arr[h.size] = f;
    h.size++;
    int i = h.size - 1;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (h.arr[i].daysleft < h.arr[parent].daysleft) {
            swap(h.arr[i], h.arr[parent]);
            i = parent;
        }
        else break;
    }
}
// Remove and return soonest expiring food (always at index 0)

food_item RemoveTopFood(Min_heap& h) {
    food_item top = h.arr[0];           // index 0 always holds the minimum
    h.arr[0] = h.arr[h.size - 1];      // move last element to top
    h.size--;
    heapify_min(h, 0);                  // fix heap downward from top
    return top;
}

// INPUT FUNCTIONS for user to add more details

void input_more_areas(Max_heap& h, int count, int areaNumber) {
    for (int i = 0; i < count; i++) {
        community c;
    }
}

```

```

    cout << "\n >>> Area " << areaNumber << " <<<\n\n";
    cout << "  Enter Area Name          : ";
    cin.ignore();
    getline(cin, c.c_name);
    cout << "  Enter Poverty Level (1-10) : ";
    cin >> c.poverty_level;

    while (c.poverty_level < 1 || c.poverty_level > 10) {
        cout << "\n  Invalid! Enter between 1 and 10 : ";
        cin >> c.poverty_level;
    }

    insert_area(h, c);
    cout << "  Area added successfully!\n";
    areaNumber++;
}
}

void input_more_food(Min_heap& h, int count, int foodNumber) {
    for (int i = 0; i < count; i++) {
        food_item f;
        cout << "\n >>> Food Item " << foodNumber << " <<<\n\n";
        cout << "  Enter Food Name          : ";
        cin.ignore();
        getline(cin, f.f_name);
        cout << "  Enter Days Left Before Expiry : ";
        cin >> f.daysleft;

        while (f.daysleft < 1) {
            cout << "\n  Invalid! Must be at least 1 day : ";
            cin >> f.daysleft;
        }

        insert_food(h, f);
        cout << "  Food item added successfully!\n";
        foodNumber++;
    }
}

// DISPLAY FUNCTIONS

void display_areas(Max_heap h) {
    // use a copy so original stays intact
    cout << "\n----- \n";
    cout << "          AREAS LOADED          \n";
    cout << " ----- \n";
    Max_heap temp = h;

```



```

int areaNumber = 6;

// ASK USER TO ADD MORE AREAS
string choice;
cout << "\nDo you want to add more areas? (yes/no) : ";
cin >> choice;
while (choice == "yes" || choice == "Yes") {
    int more;
    cout << "How many areas to add? : ";
    cin >> more;
    input_more_areas(areaHeap, more, areaNumber);
    areaNumber += more;
    display_areas(areaHeap);
    cout << "\nDo you want to add more areas? (yes/no) : ";
    cin >> choice;
}
// HARDCODED FOOD ITEMS INTO MIN HEAP
Min_heap foodheap;
foodheap.size = 0;

food_item f1; f1.f_name = "Beans";    f1.daysleft = 30;
food_item f2; f2.f_name = "Apples";   f2.daysleft = 5;
food_item f3; f3.f_name = "Bread";    f3.daysleft = 2;
food_item f4; f4.f_name = "Lentils";  f4.daysleft = 50;
food_item f5; f5.f_name = "Fresh Milk"; f5.daysleft = 1;

insert_food(foodheap, f1);
insert_food(foodheap, f2);
insert_food(foodheap, f3);
insert_food(foodheap, f4);
insert_food(foodheap, f5);

display_food(foodheap);
// foodNumber starts at 6 since 5 already inserted
int foodNumber = 6;
// ASK USER TO ADD MORE FOOD ITEMS
cout << "\nDo you want to add more food items? (yes/no) : ";
cin >> choice;

while (choice == "yes" || choice == "Yes") {
    int more;
    cout << "How many food items to add? : ";
    cin >> more;
    input_more_food(foodheap, more, foodNumber);
    foodNumber += more;
    display_food(foodheap);
}

```

```

cout << "\nDo you want to add more food items? (yes/no) : ";
cin >> choice;
}
// copies so originals stay intact for combined output
Max_heap areaheap_copy = areaHeap;
Min_heap foodheap_copy = foodheap;
// OUTPUT 1: REQ 3
cout << "\n!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n";
cout << "      FOOD DISTRIBUTION ORDER \n";
cout << " (Highest Poverty Area Served First) \n";
cout << "!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n\n";

int order = 1;
while (areaheap_copy.size > 0) {
    community c = removeTopArea(areaheap_copy);
    cout << " " << order << ". " << c.c_name;
    cout << " (Poverty Score: " << c.poverty_level << "/10)\n";
    order++;
}
// OUTPUT 2: REQ 4
cout << "\n-----\n";
cout << "      PERISHABLE ITEM ORDER \n";
cout << " (Expires Soonest = Distributed First) \n";
cout << "\n-----\n\n";

int rank = 1;
while (foodheap_copy.size > 0) {
    food_item f = RemoveTopFood(foodheap_copy);
    cout << " " << rank << ". " << f.f_name;
    cout << " (Expires in: " << f.daysleft << " days)\n";
    rank++;
}

// OUTPUT 3: COMBINED DELIVERY (poorest area+most perishable food)
cout << "\n-----\n";
cout << "      COMBINED DELIVERY PLAN \n";
cout << " (Poorest Area + Most Perishable Food) \n";
cout << "\n-----\n\n";

int delivery = 1;
while (areaHeap.size > 0 && foodheap.size > 0) {
    community c = removeTopArea(areaHeap);
    food_item f = RemoveTopFood(foodheap);
    cout << " Delivery " << delivery << ":";
    cout << "   Area : " << c.c_name
        << " (Poverty Score: " << c.poverty_level << "/10)\n";
    cout << "           Food : " << f.f_name

```

```

        << " (Expires in: " << f.daysleft << " days)\n\n";
    delivery++;
}
return 0;
}

```

OUTPUTS:

- Displaying underprivilege areas and asking user to add more

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
      WELCOME TO FOOD DISTRIBUTION ORDER
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

-----
                AREAS LOADED
-----

1. Lyari   (Poverty Level: 8/10)
2. Nazimabad (Poverty Level: 7/10)
3. Orangi  (Poverty Level: 6/10)
4. Malir   (Poverty Level: 5/10)
5. Korangi (Poverty Level: 4/10)

Do you want to add more areas? (yes/no) : yes
How many areas to add? : 1

>>> Area 6 <<<

Enter Area Name      : Saddar
Enter Poverty Level (1-10) : 9
Area added successfully!

-----
                AREAS LOADED
-----

1. Saddar (Poverty Level: 9/10)
2. Lyari  (Poverty Level: 8/10)
3. Nazimabad (Poverty Level: 7/10)
4. Orangi  (Poverty Level: 6/10)
5. Malir   (Poverty Level: 5/10)
6. Korangi (Poverty Level: 4/10)

Do you want to add more areas? (yes/no) : no

```

- Displaying available food items to deliver according to their expiry days and asking user if he wants to add more

```
-----  
                        FOOD ITEMS LOADED  
-----  
1. Fresh Milk (Expires in: 1 days)  
2. Bread (Expires in: 2 days)  
3. Apples (Expires in: 5 days)  
4. Beans (Expires in: 30 days)  
5. Lentils (Expires in: 50 days)  
  
Do you want to add more food items? (yes/no) : yes  
How many food items to add? : 1  
  
>>> Food Item 6 <<<  
  
Enter Food Name : Wheat flour  
Enter Days Left Before Expiry : 40  
Food item added successfully!  
  
-----  
                        FOOD ITEMS LOADED  
-----  
1. Fresh Milk (Expires in: 1 days)  
2. Bread (Expires in: 2 days)  
3. Apples (Expires in: 5 days)  
4. Beans (Expires in: 30 days)  
5. Wheat flour (Expires in: 40 days)  
6. Lentils (Expires in: 50 days)  
  
Do you want to add more food items? (yes/no) : no
```


- Displaying updated Areas and Perishable Food items

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!  
      FOOD DISTRIBUTION ORDER  
(Highest Poverty Area Served First)  
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

1. Saddar (Poverty Score: 9/10)
2. Lyari (Poverty Score: 8/10)
3. Nazimabad (Poverty Score: 7/10)
4. Orangi (Poverty Score: 6/10)
5. Malir (Poverty Score: 5/10)
6. Korangi (Poverty Score: 4/10)

```
-----  
      PERISHABLE ITEM ORDER  
(Expires Soonest = Distributed First)  
-----
```

1. Fresh Milk (Expires in: 1 days)
 2. Bread (Expires in: 2 days)
 3. Apples (Expires in: 5 days)
 4. Beans (Expires in: 30 days)
 5. Wheat flour (Expires in: 40 days)
 6. Lentils (Expires in: 50 days)
- ```

```

- **Displaying combined delivery plan according to poverty level of area and expiry days of food item**

-----  
COMBINED DELIVERY PLAN

(Poorest Area + Most Perishable Food)  
-----

|             |                                                                                |
|-------------|--------------------------------------------------------------------------------|
| Delivery 1: | Area : Saddar (Poverty Score: 9/10)<br>Food : Fresh Milk (Expires in: 1 days)  |
| Delivery 2: | Area : Lyari (Poverty Score: 8/10)<br>Food : Bread (Expires in: 2 days)        |
| Delivery 3: | Area : Nazimabad (Poverty Score: 7/10)<br>Food : Apples (Expires in: 5 days)   |
| Delivery 4: | Area : Orangi (Poverty Score: 6/10)<br>Food : Beans (Expires in: 30 days)      |
| Delivery 5: | Area : Malir (Poverty Score: 5/10)<br>Food : Wheat flour (Expires in: 40 days) |
| Delivery 6: | Area : Korangi (Poverty Score: 4/10)<br>Food : Lentils (Expires in: 50 days)   |

## TASK: Time complexity Analysis

### Task 1 Binary Search Tree (BST)

In Task 1, the BST stores community areas sorted automatically by their poverty\_level. The BST is also used to store and display suppliers and warehouses through simple array. The following functions are: create\_node(), insert(), search(), In\_Order(), display\_suppliers(), display\_warehouses(), display\_communities(), input\_supplier(), input\_warehouse(), and input\_community().

#### create\_node() Function

This function creates a new BST node and sets its data.

```
BST_Node* create_node(community c) { //community c = data to be inserted

 BST_Node* temp = new BST_Node();

 temp->data = c;

 temp->left = NULL;

 temp->right = NULL;

 return temp;

}
```

#### Step by step Cost Analysis:

| Statement                       | Cost | Times Executed |
|---------------------------------|------|----------------|
| BST_Node* temp = new BST_Node() | c1   | 1              |
| temp->data = c                  | c2   | 1              |
| temp->left = NULL               | c3   | 1              |
| temp->right = NULL              | c4   | 1              |
| return temp                     | c5   | 1              |

#### Total Cost Calculation:

Total Cost = c1 + c2 + c3 + c4 + c5

= Ignoring constants

= O(1)

| Case         | Condition              | Time |
|--------------|------------------------|------|
| Best Case    | Always same operations | O(1) |
| Average Case | Always same operations | O(1) |
| Worst Case   | Always same operations | O(1) |

**Time Complexity of create\_node() = O(1)**

## insert() Function:

This function inserts a new community into the BST based on its poverty\_level. It goes left if the new level is smaller, right if larger, and also right if equal (to handle duplicates without crashing).

```
BST_Node* insert(BST_Node* root, community c) {
 if (root == NULL) {
 BST_Node* root = create_node(c);
 return root;
 }
 if (c.poverty_level < root->data.poverty_level) {
 root->left = insert(root->left, c);
 return root;
 }
 if (c.poverty_level > root->data.poverty_level) {
 root->right = insert(root->right, c);
 return root;
 }
}
```

### Step by step Cost Analysis:

| Statement                                       | Cost | Times Executed |
|-------------------------------------------------|------|----------------|
| if (root == NULL)                               | c1   | 1              |
| create_node(c) and return root                  | c2   | 1              |
| if (c.poverty_level < root->data.poverty_level) | c3   | 1              |
| root->left = insert(root->left, c);             | c4   | 1              |
| if (c.poverty_level > root->data.poverty_level) | c5   | 1              |
| root->right = insert(root->right, c);           | c6   | 1              |

### Total Cost Calculation:

#### For h (height of tree after inserting n node): h

Total Cost = (c1 + c3 + c4) x h + c2

Ignoring constants

$$= O(h)$$

For a balanced BST with n nodes:

We know,

$$\text{height } h = \log_2(n+1) + 1$$

after ignoring constants,

$$\text{Total Cost} = O(\log n)$$

| Case         | Condition                                             | Time        |
|--------------|-------------------------------------------------------|-------------|
| Best Case    | Tree is empty new node inserted at root immediately   | $O(1)$      |
| Average Case | Tree is balanced height = $\log_2(n)$                 | $O(\log n)$ |
| Worst Case   | All nodes inserted in sorted order straight line tree | $O(n)$      |

**Time Complexity of insert() =  $O(\log n)$  average case**

## Why height becomes log n

$$\text{nodes} = 2^{(h+1)} - 1$$

Now solve for h:

$$n = 2^{(h+1)} - 1$$

$$n + 1 = 2^{(h+1)}$$

Take  $\log_2$  to solve (h+1) in power:

$$\log_2(n+1) = \log_2(2^{(h+1)})$$

we know that  $\log_2(2) = 1$ .

$$\log_2(n+1) = h + 1$$

So:

$$h = \log_2(n+1) + 1$$

Ignoring constants in Big O:

$$h = \log n$$

## search() Function

This function searches the BST by poverty\_level. When a match is found, it prints the area details

**Code from Task 1:**

```
BST_Node* search(BST_Node* root, int p_level_to_find) {
 if (root == NULL) {
 cout << "\nNo Area found with poverty level: " << p_level_to_find << "\n\n";
 return NULL;
 }
 if (p_level_to_find == root->data.poverty_level) {
 return root;
 }
 else if (p_level_to_find < root->data.poverty_level) {
 return search(root->left, p_level_to_find);
 }
 else {
 return search(root->right, p_level_to_find);
 }
}
```

**Step by step Cost Analysis:**

| Statement                                               | Cost | Times Executed |
|---------------------------------------------------------|------|----------------|
| if (root == NULL)                                       | c1   | 1              |
| return NULL                                             | c2   | 1              |
| if (p_level_to_find == root<br>>data.poverty_level)     | C3   | 1              |
| return root;                                            | C4   | 1              |
| else if (p_level_to_find < root<br>>data.poverty_level) | C5   | 1              |
| return search(root >left, p_level_to_find)              | C6   | 1              |
| else                                                    | C7   | 1              |
| return search(root >right, p_level_to_find)             | C8   | 1              |

**Total Cost Calculation:**

Total Cost = (c1 + c3 + c5 + c4+c5+c5+c6+c7+c8) x h (If the tree has height h)  
 = constant x h

After ignoring constants:

$$= O(h)$$

We know  $h = \log n$ ,

$$= O(\log n) \text{ for balanced BST}$$

| Case         | Condition                                 | Time        |
|--------------|-------------------------------------------|-------------|
| Best Case    | Match found at root first node checked    | $O(1)$      |
| Average Case | Match found somewhere in balanced tree    | $O(\log n)$ |
| Worst Case   | Match at deepest leaf or not found at all | $O(\log n)$ |

**Time Complexity of search() =  $O(\log n)$**

## In\_Order() Function

This function performs an in order traversal of the BST visiting left subtree, then root, then right subtree. This produces output sorted from lowest to highest poverty\_level automatically.

**Code from Task 1:**

```
void In_Order(BST_Node* root) {
 if (root != NULL) {
 In_Order(root->left);
 cout << "\nArea: " << root->data.area_name;
 cout << "\nPopulation: " << root->data.population;
 cout << "\nFood Required: " << root->data.food_required << " kg";
 cout << "\nPoverty Level: " << root->data.poverty_level << "/10\n";
 cout << "\n \n";
 In_Order(root->right);
 }
}
```

**Step by step Cost Analysis:**

| Statement             | Cost | Times Executed                        |
|-----------------------|------|---------------------------------------|
| if (root != NULL)     | c1   | n+1 (n nodes + NULL checks at leaves) |
| In_Order(root->left)  | c2   | n                                     |
| cout area_name        | c3   | n                                     |
| cout population       | c4   | n                                     |
| cout food_required    | c5   | n                                     |
| cout poverty_level    | c6   | n                                     |
| In_Order(root->right) | c7   | n                                     |

**Total Cost Calculation:**

Every single node is visited exactly once. There are  $n$  nodes total.

$$\begin{aligned}\text{Total Cost} &= (n+1) \times c_1 + n \times c_2 + n \times c_3 + n \times c_4 + n \times c_5 + n \times c_6 + n \times c_7 \\ &= n \times (c_1 + c_2 + c_3 + c_4 + c_5 + c_6 + c_7) + c_1\end{aligned}$$

Ignoring constants.

$$= n$$

$$= O(n)$$

| Case         | Condition           | Time   |
|--------------|---------------------|--------|
| Best Case    | Only 1 node in tree | $O(1)$ |
| Average Case | $n$ nodes           | $O(n)$ |
| Worst Case   | $n$ nodes           | $O(n)$ |

**Time Complexity of In\_Order() =  $O(n)$**

**Note: In\_Order is used only for display purposes. It does not violate the  $O(\log n)$  requirement because core data operations (insert, search) remain  $O(\log n)$ .**

## display\_suppliers() Function

This function loops through the suppliers array and prints each supplier's name, type, and food available.

**Code from Task 1:**

```
void display_suppliers(supplier suppliers[], int count) {
 cout << "\n ----- \n";
 cout << " TOTAL SUPPLIERS \n";
 cout << " ----- \n";
 for (int i = 0; i < count; i++) {
 cout << " " << i + 1 << ". " << suppliers[i].s_name
 << " [" << suppliers[i].type << "]"
 << " -- " << suppliers[i].food_available << " kg available\n";
 }
}
```

**Step by step Cost Analysis:**

| Statement                       | Cost  | Times Executed |
|---------------------------------|-------|----------------|
| cout lines                      | $c_1$ | 1              |
| for (int i = 0; i < count; i++) | $c_2$ | $n + 1$        |
| Inside for loop                 | $c_3$ | $n$            |

**Total Cost Calculation:**

$$\text{Total Cost} = c_1 + (n+1) \times c_2 + nc_3$$

Ignoring constants



$$= n$$

$$= O(n)$$

| Case         | Condition | Time   |
|--------------|-----------|--------|
| Best Case    | 0         | $O(1)$ |
| Average Case | $n$       | $O(n)$ |
| Worst Case   | $n$       | $O(n)$ |

**Time Complexity of display\_suppliers() =  $O(n)$**

## display\_warehouses() Function:

Same structure as display\_suppliers.

**Time Complexity of display\_warehouses() =  $O(n)$**

## display\_communities() Function:

```
void display_communities(BST_Node* root) {
 cout << "\n ----- \n";
 cout << " UNDERPRIVILEGED AREAS \n"; //(BST Sorted)
 cout << " Order: Lowest to Highest Poverty \n";
 cout << " ----- \n";
 In_Order(root);
}
```

### Step by step Cost Analysis:

| Statement       | Cost | Times Executed |
|-----------------|------|----------------|
| cout statements | c1   | 1              |
| In_Order(root)  | c2   | $O(n)$         |

### Total Cost Calculation:

Total Cost =  $c1 + O(n)$

$$= O(n)$$

**Time Complexity of display\_communities() =  $O(n)$**

## input\_supplier() Function

```
void input_supplier(supplier suppliers[], int startIndex, int count) {
 for (int i = startIndex; i < startIndex + count; i++) {
```

```

 cout << "\n >>> Supplier " << i + 1 << " <<<\n\n";

 cout << " Enter Name : ";

 cin.ignore();

 getline(cin, suppliers[i].s_name);

 cout << " Enter type (Restaurant / Farm / Supermarket) : ";

 getline(cin, suppliers[i].type);

 cout << " Food Available (kg) : ";

 cin >> suppliers[i].food_available;

}

}

```

### Step by step Cost Analysis:

| Statement                                             | Cost | Times Executed |
|-------------------------------------------------------|------|----------------|
| for (int i = startIndex; i < startIndex + count; i++) | c1   | n + 1          |
| Count statements. inside for loop.                    | c2   | n              |

### Total Cost Calculation:

Total Cost = (n+1)xc1 + n x c2

$$= c1 + nC1 + nC2$$

$$= n(c1 + c2)$$

$$= O(n)$$

Time Complexity of input\_supplier() = O(n)

### input\_warehouse() Function:

Same as input\_supplier().

Time Complexity of input\_warehouse() = O(n)

### input\_community() Function:

```

BST_Node* input_community(BST_Node* root, int count, int communityNumber) {

```

```

 for (int i = 0; i < count; i++) {

 community c;

 cout << "\n >>> Community " << communityNumber << " <<<\n\n";

 cout << " Enter Area Name : ";

 cin.ignore();

 getline(cin, c.area_name);
 }
}

```

```

 cout << " Enter Population : ";

 cin >> c.population;

 cout << " Enter Food Required (kg/day) : ";

 cin >> c.food_required;

 cout << " Enter Poverty Level (1 to 10) : ";

 cin >> c.poverty_level;

 while (c.poverty_level < 1 || c.poverty_level > 10) {

 cout << "\n Invalid! Enter between 1 and 10 : ";

 cin >> c.poverty_level;

 }

 root = insert(root, c);

 cout << " Community added successfully!\n";

 communityNumber++; // increment for next community

}

return root;
}

```

### Step by step Cost Analysis:

| Statement                                           | Cost | Times Executed |
|-----------------------------------------------------|------|----------------|
| for (int i = 0; i < count; i++)                     | c1   | n + 1          |
| cin/cout statements                                 | c2   | n              |
| while (c.poverty_level < 1    c.poverty_level > 10) | c3   | 1              |
| root = insert(root, c)                              | c4   | n x O(log n)   |
| communityNumber++                                   | c5   | n              |
| return root                                         | c6   | 1              |

### Total Cost Calculation:

$$\begin{aligned}
 \text{Total Cost} &= (n+1) \times c1 + n \times c2 + n \times c3 + n \times O(\log n) + n \times c5 \\
 &= n \times (c1+c2+c3+c5) + n \times O(\log n) \\
 &= O(n) + O(n \log n) \\
 &= O(n \log n)
 \end{aligned}$$

| Case         | Condition                     | Time          |
|--------------|-------------------------------|---------------|
| Best Case    | 1 community, inserted at root | $O(\log n)$   |
| Average Case | n communities, balanced tree  | $O(n \log n)$ |
| Worst Case   | n communities, skewed tree    | $O(n^2)$      |

**Time Complexity of input\_community() =  $O(n \log n)$**

**Note: input\_community() is used only for input purposes. It does not violate the  $O(\log n)$  requirement because core data operations (insert, search) remain  $O(\log n)$ .**

## Task 1 Overall Time Complexities

| Function              | Data Structure Used | Best Case   | Average Case  | Worst Case  |
|-----------------------|---------------------|-------------|---------------|-------------|
| create_node()         | BST Node            | $O(1)$      | $O(1)$        | $O(1)$      |
| insert()              | BST                 | $O(1)$      | $O(\log n)$   | $O(n)$      |
| search()              | BST                 | $O(1)$      | $O(\log n)$   | $O(\log n)$ |
| In_Order()            | BST                 | $O(1)$      | $O(n)$        | $O(n)$      |
| display_suppliers()   | Array               | $O(1)$      | $O(n)$        | $O(n)$      |
| display_warehouses()  | Array               | $O(1)$      | $O(n)$        | $O(n)$      |
| display_communities() | BST via In_Order    | $O(1)$      | $O(n)$        | $O(n)$      |
| input_supplier()      | Array               | $O(1)$      | $O(n)$        | $O(n)$      |
| input_warehouse()     | Array               | $O(1)$      | $O(n)$        | $O(n)$      |
| input_community()     | BST + Array         | $O(\log n)$ | $O(n \log n)$ | $O(n^2)$    |

The assignment requirement states time complexity must not exceed  $O(\log n)$ . Both insert() and search() satisfy this requirement in the average case, which is the realistic case since poverty levels are entered in random order. Display and input functions are  $O(n)$  or  $O(n \log n)$  but these are not core data structure operations.

## TASK 2

### Max Heap and Min Heap

In Task 2, Max\_heap stores community areas and ensures the highest poverty\_level is always served first (Requirement 3). Min\_heap stores food items and ensures the lowest daysleft (most perishable) is always distributed first (Requirement 4). Both heaps are stored as plain arrays inside a struct. The following functions are analyzed: heapify\_max(), build\_max\_heap(), insert\_area(), removeTopArea(), heapify\_min(), build\_min\_heap(), insert\_food(), RemoveTopFood(), input\_more\_areas(), input\_more\_food(), display\_areas(), display\_food(), and the three output loops in main().

### heapify\_max() Function

This function fixes the Max Heap downward from index i. It checks the left and right children and swaps with the largest one if needed, then recurses down.

**Code from Task 2:**

```
void heapify_max(Max_heap& h, int i) {
```

```

int largest = i;

int left = 2 * i + 1;

int right = 2 * i + 2;

// if left child exists and is greater than current largest
if (left < h.size && h.arr[left].poverty_level > h.arr[largest].poverty_level)

 largest = left;

// if right child exists and is greater than current largest
if (right < h.size && h.arr[right].poverty_level > h.arr[largest].poverty_level)

 largest = right;

// if largest is not current index, swap and continue fixing downward
if (largest != i) {

 swap(h.arr[i], h.arr[largest]);

 heapify_max(h, largest);

}
}

```

#### Step by step Cost Analysis:

| Statement                       | Cost | Times Executed |
|---------------------------------|------|----------------|
| int largest = i                 | c1   | 1              |
| int left = 2*i+1                | c2   | 1              |
| int right = 2*i+2               | c3   | 1              |
| if left child check             | c4   | 1              |
| largest = left                  | c5   | 1              |
| if right child check            | c6   | 1              |
| largest = right                 | c7   | 1              |
| if (largest != i)               | c8   | 1              |
| swap(h.arr[i], h.arr[largest])  | c9   | 1              |
| heapify_max(h, largest) recurse | c10  | h              |

#### Total Cost Calculation:

A heap of  $n$  elements has height  $h = \log_2(n)$ . heapify\_max goes down at most  $h$  levels from the starting index to a leaf.

$$\text{Total Cost} = c1 + c2 + c3 + c4 + c5 + c6 + c7 + c8 + c9 \times h$$

We know,  $h = \log_2(n+1) - 1$

Ignoring constants in Big O:

$$h = \log_2(n)$$

$$\text{Total cost} = O(\log n)$$

| Case         | Condition                                                 | Time        |
|--------------|-----------------------------------------------------------|-------------|
| Best Case    | Element already in correct position no swap, no recursion | $O(1)$      |
| Average Case | Element goes down a few levels                            | $O(\log n)$ |
| Worst Case   | Element goes all the way down to a leaf                   | $O(\log n)$ |

**Time Complexity of heapify\_max() =  $O(\log n)$**

## build\_max\_heap() Function

This function builds the entire Max Heap from an unsorted array all at once. It starts from the last non leaf node and calls heapify\_max downward for each node.

**Code from Task 2:**

```
void build_max_heap(Max_heap& h) {
 for (int i = (h.size / 2) - 1; i >= 0; i--)
 heapify_max(h, i);
}
```

### Step by step Cost Analysis:

- many nodes take only 1 step
- some take 2 steps
- very few take  $\log n$  steps

| Statement                                   | Cost | Times Executed       |
|---------------------------------------------|------|----------------------|
| for (int i = (h.size / 2) - 1; i >= 0; i--) | c1   | n/2                  |
| heapify_max(h, i)                           | c2   | $O(\log n)$ per call |

### Total Cost Calculation:

estimate:  $(n/2) \times O(\log n) = O(n \log n)$

Most nodes are near the bottom and take  $O(1)$  or  $O(2)$  steps.

We write:  $O(n)$  build\_max\_heap is a linear time operation

| Case         | Condition                | Time   |
|--------------|--------------------------|--------|
| Best Case    | Already a valid max heap | $O(n)$ |
| Average Case | Random unsorted data     | $O(n)$ |
| Worst Case   | Reverse sorted data      | $O(n)$ |

**Time Complexity of build\_max\_heap() =  $O(n)$**

## insert\_area() Function

This function inserts a new community into the Max Heap. It places the element at the end of the array and sets it up by comparing with its parent until the max heap property is satisfied.

**Code from Task 2:**

```
void insert_area(Max_heap& h, community c) {
 h.arr[h.size] = c;
 h.size++;
 int i = h.size - 1;
```

```

while (i > 0) {
 int parent = (i - 1) / 2;
 if (h.arr[parent].poverty_level < h.arr[i].poverty_level) {
 swap(h.arr[i], h.arr[parent]);
 i = parent;
 }
 else break;
}
}

```

#### Step by step Cost Analysis:

| Statement                      | Cost | Times Executed            |
|--------------------------------|------|---------------------------|
| h.arr[h.size] = c              | c1   | 1                         |
| h.size++                       | c2   | 1                         |
| int i = h.size - 1             | c3   | 1                         |
| while (i > 0)                  | c4   | h+1 where $h = \log_2(n)$ |
| int parent = (i - 1) / 2       | c5   | h                         |
| if comparison of poverty_level | c6   | h                         |
| swap(h.arr[i], h.arr[parent])  | c7   | h                         |
| i = parent                     | c8   | h                         |
| break                          | c9   | 1                         |

#### Total Cost Calculation:

Total Cost =  $c1 + c2 + c3 + (h+1) \times c4 + h \times c5 + h \times c6 + h \times c7 + h \times c8$

$$= h \times (c4 + c5 + c6 + c7 + c8)$$

$$= \log(n)$$

$$= O(\log n)$$

| Case         | Condition                                                                 | Time        |
|--------------|---------------------------------------------------------------------------|-------------|
| Best Case    | New element has smallest poverty_level no swap needed, breaks immediately | $O(1)$      |
| Average Case | Element goes up a few levels                                              | $O(\log n)$ |
| Worst Case   | New element has highest poverty_level all the way to root                 | $O(\log n)$ |

**Time Complexity of insert\_area() =  $O(\log n)$**

## removeTopArea() Function

This function removes and returns the community with the highest poverty\_level (always at index 0). It replaces the top with the last element, reduces size, and calls heapify\_max to fix the heap downward.

#### Code from Task 2:

```

community removeTopArea(Max_heap& h) {

```

```

community top = h.arr[0];

h.arr[0] = h.arr[h.size - 1];

h.size--;

heapify_max(h, 0);

return top;

}

```

### Step by step Cost Analysis:

| Statement                    | Cost | Times Executed                       |
|------------------------------|------|--------------------------------------|
| community top = h.arr[0]     | c1   | 1                                    |
| h.arr[0] = h.arr[h.size - 1] | c2   | 1                                    |
| h.size--                     | c3   | 1                                    |
| heapify_max(h, 0) call       | c4   | 1 call, but heapify runs $O(\log n)$ |
| return top                   | c5   | 1                                    |

### Total Cost Calculation:

Total Cost =  $c1 + c2 + c3 + \text{heapify\_max cost} + c5$   
 = constants +  $O(\log n)$   
 =  $O(\log n)$

| Case         | Condition                                                     | Time        |
|--------------|---------------------------------------------------------------|-------------|
| Best Case    | Last element placed at top is already largest no swaps needed | $O(1)$      |
| Average Case | Element moves down a few levels                               | $O(\log n)$ |
| Worst Case   | Element moves all the way down to a leaf                      | $O(\log n)$ |

**Time Complexity of removeTopArea() =  $O(\log n)$**

## heapify\_min() Function

This function fixes the Min Heap downward from index i. It checks children and swaps with the smallest one if the current element is larger, then recurses.

### Code from Task 2:

```

void heapify_min(Min_heap& h, int i) {

 int smallest = i;

 int left = 2 * i + 1;

 int right = 2 * i + 2;

 // if left child exists and is smaller than current smallest
 if (left < h.size && h.arr[left].daysleft < h.arr[smallest].daysleft)
 smallest = left;

 // if right child exists and is smaller than current smallest
 if (right < h.size && h.arr[right].daysleft < h.arr[smallest].daysleft)

```



```

 smallest = right;

 // if smallest is not current index, swap and continue fixing downward
 if (smallest != i) {
 swap(h.arr[smallest], h.arr[i]);
 heapify_min(h, smallest);
 }
}

```

#### Step by step Cost Analysis:

| Statement                        | Cost | Times Executed |
|----------------------------------|------|----------------|
| int smallest = i                 | c1   | 1              |
| int left = 2*i+1                 | c2   | 1              |
| int right = 2*i+2                | c3   | 1              |
| if left child check (daysleft)   | c4   | 1              |
| smallest = left                  | c5   | 1              |
| if right child check (daysleft)  | c6   | 1              |
| smallest = right                 | c7   | 1              |
| if (smallest != i)               | c8   | 1              |
| swap(h.arr[smallest], h.arr[i])  | c9   | 1              |
| heapify_min(h, smallest) recurse | c10  | h              |

#### Total Cost Calculation:

Total Cost =  $c1+c2+c3+c4+c5+c6+c7+c8+c9 \times h$

Total Cost = constant  $\times \log_2(n)$

=  $O(\log n)$

| Case         | Condition                                      | Time        |
|--------------|------------------------------------------------|-------------|
| Best Case    | Element already smallest no swap, no recursion | $O(1)$      |
| Average Case | Element goes down a few levels                 | $O(\log n)$ |
| Worst Case   | Element goes all the way to a leaf             | $O(\log n)$ |

Time Complexity of heapify\_min() =  $O(\log n)$

## build\_min\_heap() Function

Same structure as build\_max\_heap. Starts from the last non leaf node and calls heapify\_min for each node going upward.

#### Code from Task 2:

```

void build_min_heap(Min_heap& h) {
 for (int i = (h.size) / 2 - 1; i >= 0; i--)
 heapify_min(h, i);
}

```

#### Step by step Cost Analysis:

| Statement                                   | Cost | Times Executed    |
|---------------------------------------------|------|-------------------|
| for (int i = (h.size / 2) - 1; i >= 0; i--) | c1   | n/2               |
| heapify_min(h, i);                          | c2   | O(log n) per call |

### Explanation:

| Heap Level      | Number of Nodes | Heapify Work |
|-----------------|-----------------|--------------|
| Bottom Level    | n/2             | O(1)         |
| One Level Above | n/4             | O(2)         |
| Next Level      | n/8             | O(3)         |
| Top Levels      | very few        | O(log n)     |

This complete series simplifies to:

O(n)

#### Total Cost Calculation:

Same analysis as build\_max\_heap.

O(n)

**Time Complexity of build\_min\_heap() = O(n)**

## insert\_food() Function

This function inserts a new food item into the Min Heap. It places at end and by comparing daysleft smaller daysleft goes higher (soonest expiry at top).

#### Code from Task 2:

```
void insert_food(Min_heap& h, food_item f) {
 h.arr[h.size] = f;
 h.size++;
 int i = h.size - 1;
 while (i > 0) {
 int parent = (i - 1) / 2;
 if (h.arr[i].daysleft < h.arr[parent].daysleft) {
 swap(h.arr[i], h.arr[parent]);
 i = parent;
 }
 else break;
 }
}
```

#### Step by step Cost Analysis:

| Statement                     | Cost | Times Executed                     |
|-------------------------------|------|------------------------------------|
| h.arr[h.size] = f             | c1   | 1                                  |
| h.size++                      | c2   | 1                                  |
| int i = h.size - 1            | c3   | 1                                  |
| while (i > 0)                 | c4   | h+1 where h = log <sub>2</sub> (n) |
| int parent = (i - 1)/2        | c5   | h                                  |
| if comparison of daysleft     | c6   | h                                  |
| swap(h.arr[i], h.arr[parent]) | c7   | h                                  |
| i = parent                    | c8   | h                                  |
| break                         | c9   | 1                                  |

**Total Cost Calculation:**

$$\begin{aligned}
 \text{Total Cost} &= c1 + c2 + c3 + (h+1) \times c4 + h \times c5 + h \times c6 + h \times c7 + h \times c8 \\
 &= h \times (c4 + c5 + c6 + c7 + c8) \\
 &= \log(n) \\
 &= O(\log n)
 \end{aligned}$$

| Case         | Condition                                               | Time     |
|--------------|---------------------------------------------------------|----------|
| Best Case    | New food has most days left no swap, breaks immediately | O(1)     |
| Average Case | Food bubbles up a few levels                            | O(log n) |
| Worst Case   | New food expires soonest bubbles all the way to root    | O(log n) |

**Time Complexity of insert\_food() = O(log n)**

## RemoveTopFood() Function

This function removes and returns the food item with the fewest daysleft (always at index 0). It replaces top with last element and calls heapify\_min to fix downward.

**Code from Task 2:**

```

food_item RemoveTopFood(Min_heap& h) {
 food_item top = h.arr[0];
 h.arr[0] = h.arr[h.size - 1];
 h.size--;
 heapify_min(h, 0);
 return top;
}

```

**Step by step Cost Analysis:**

| Statement                    | Cost | Times Executed |
|------------------------------|------|----------------|
| food_item top = h.arr[0]     | c1   | 1              |
| h.arr[0] = h.arr[h.size - 1] | c2   | 1              |
| h.size--                     | c3   | 1              |
| heapify_min(h, 0) call       | c4   | O(log n)       |
| return top                   | c5   | 1              |

**Total Cost Calculation:**

$$\begin{aligned}
 \text{Total Cost} &= c_1 + c_2 + c_3 + \text{heapify\_min cost} + c_5 \\
 &= \text{constants} + O(\log n) \\
 &= O(\log n)
 \end{aligned}$$

| Case         | Condition                                 | Time        |
|--------------|-------------------------------------------|-------------|
| Best Case    | Last element is already smallest no swaps | $O(1)$      |
| Average Case | Element moves down a few levels           | $O(\log n)$ |
| Worst Case   | Element moves all the way to leaf         | $O(\log n)$ |

**Time Complexity of RemoveTopFood() =  $O(\log n)$**

## display\_areas() Function

Displays all areas currently in the Max Heap from highest to lowest poverty. It uses a temp copy so the original heap stays intact, then calls removeTopArea n times.

**Code from Task 2:**

```

void display_areas(Max_heap h) {
 // use a copy so original stays intact

 cout << "\n ----- \n";

 cout << " AREAS LOADED \n";

 cout << " ----- \n";

 Max_heap temp = h;

 int rank = 1;

 // show all from highest to lowest using a temp copy

 while (temp.size > 0) {
 community c = removeTopArea(temp);

 cout << " " << rank << ". " << c.c_name << " (Poverty Level: " << c.poverty_level << "/10)\n";

 rank++;
 }
}

```

**Total Cost Calculation:**

Copy heap:

$O(n)$  copying n elements

While loop runs n times

removeTopArea each:  $O(\log n)$

$$\begin{aligned}
 \text{Total Cost} &= O(n) + n \times O(\log n) \\
 &= O(n) + O(n \log n) \\
 &= O(n \log n) \quad (\text{dominant term})
 \end{aligned}$$

**Time Complexity of display\_areas() =  $O(n \log n)$**

## display\_food() Function

Same structure as display\_areas. Uses a temp copy of Min Heap and calls RemoveTopFood n times to display food items from soonest to latest expiry.

**Code from task 2:**

```

void display_food(Min_heap h) {

 cout << "\n ----- \n";
 cout << " FOOD ITEMS LOADED \n";
 cout << " ----- \n";

 Min_heap temp = h;

 int rank = 1;

 while (temp.size > 0) {

 food_item f = RemoveTopFood(temp);

 cout << " " << rank << ". " << f.f_name << " (Expires in: " << f.daysleft << " days)\n";

 rank++;

 }

}

```

**Total Cost Calculation:**

$$\begin{aligned}
 \text{Total Cost} &= O(n) + n \times O(\log n) \\
 &= O(n \log n)
 \end{aligned}$$

**Time Complexity of display\_food() =  $O(n \log n)$**

## input\_more\_areas() Function

This function takes input for new areas from the user and inserts each one into the Max Heap using insert\_area(). The areaNumber is used to display correct numbering

**Code from Task 2:**

```

void input_more_areas(Max_heap& h, int count, int areaNumber) {

 for (int i = 0; i < count; i++) {

 community c;

 cout << "\n >>> Area " << areaNumber << " <<< \n\n";
 }
}

```

```

 cout << " Enter Area Name : ";

 cin.ignore();

 getline(cin, c.c_name);

 cout << " Enter Poverty Level (1-10) : ";

 cin >> c.poverty_level;

 while (c.poverty_level < 1 || c.poverty_level > 10) {

 cout << "\n Invalid! Enter between 1 and 10 : ";

 cin >> c.poverty_level;

 }

 insert_area(h, c);

 cout << " Area added successfully!\n";

 areaNumber++;

}
}

```

**Step by step analysis:**

| Statement                                           | Cost | Times Executed |
|-----------------------------------------------------|------|----------------|
| for (int i = 0; i < count; i++)                     | c1   | n+1            |
| Community c;                                        | c2   | n              |
| Input/output statements                             | c3   | n              |
| while (c.poverty_level < 1    c.poverty_level > 10) | c4   | n(n+1)         |
| insert_area(h, c)                                   | c5   | n x O(log n)   |
| Cout details                                        | c6   | n x n          |
| areaNumber++                                        | c7   | n x n          |

**Total Cost Calculation:**

Total Cost = c1+ n C1+nC2 + nC3 + nC4 + n x O(log n) c5+ n<sup>2</sup> C6 + n C4<sup>2</sup> + n<sup>2</sup> C7

= O(n) + O(n log n)

= O(n log n)

**Time Complexity of input\_more\_areas() = O(n log n)**

## input\_more\_food() Function

Takes food item input and inserts each into Min Heap using insert\_food().

**code:**

```

void input_more_food(Min_heap& h, int count, int foodNumber) {
 for (int i = 0; i < count; i++) {
 food_item f;
 }
}

```

```

 cout << "\n >>> Food Item " << foodNumber << " <<<\n\n";
 cout << " Enter Food Name : ";
 cin.ignore();
 getline(cin, f.f_name);
 cout << " Enter Days Left Before Expiry : ";
 cin >> f.daysleft;

 while (f.daysleft < 1) {
 cout << "\n Invalid! Must be at least 1 day : ";
 cin >> f.daysleft;
 }

 insert_food(h, f);
 cout << " Food item added successfully!\n";
 foodNumber++;
}
}

```

| Statement                       | Cost | Times Executed |
|---------------------------------|------|----------------|
| for (int i = 0; i < count; i++) | c1   | n+1            |
| food_item f;                    | c2   | n              |
| Input/output statements         | c3   | n              |
| while (f.daysleft < 1)          | c4   | n(n+1)         |
| insert_area(h, c)               | c5   | n x O(log n)   |
| Cout details                    | c6   | n x n          |
| foodNumber++                    | c7   | n x n          |

**Total Cost Calculation:**

Total Cost =  $n^2 + n + O(n \log n) = O(n \log n)$

**Time Complexity of input\_more\_food() =  $O(n \log n)$**

## Task 2 Overall Time Complexity Summary

| Function           | Heap Type | Best Case     | Average Case  | Worst Case    |
|--------------------|-----------|---------------|---------------|---------------|
| heapify_max()      | Max Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| build_max_heap()   | Max Heap  | $O(n)$        | $O(n)$        | $O(n)$        |
| insert_area()      | Max Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| removeTopArea()    | Max Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| heapify_min()      | Min Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| build_min_heap()   | Min Heap  | $O(n)$        | $O(n)$        | $O(n)$        |
| insert_food()      | Min Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| RemoveTopFood()    | Min Heap  | $O(1)$        | $O(\log n)$   | $O(\log n)$   |
| display_areas()    | Max Heap  | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| display_food()     | Min Heap  | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |
| input_more_areas() | Max Heap  | $O(\log n)$   | $O(n \log n)$ | $O(n \log n)$ |
| input_more_food()  | Min Heap  | $O(\log n)$   | $O(n \log n)$ | $O(n \log n)$ |

## Overall Time Complexity Summary of Both Tasks

| Task   | Function              | Data Structure | Best       | Average    | Worst              |
|--------|-----------------------|----------------|------------|------------|--------------------|
| Task 1 | create_node()         | BST Node       | O(1)       | O(1)       | O(1)               |
| Task 1 | insert()              | BST            | O(1)       | O(log n)   | O(n)               |
| Task 1 | search()              | BST            | O(1)       | O(log n)   | O(log n)           |
| Task 1 | In_Order()            | BST            | O(1)       | O(n)       | O(n)               |
| Task 1 | display_suppliers()   | Array          | O(1)       | O(n)       | O(n)               |
| Task 1 | display_warehouses()  | Array          | O(1)       | O(n)       | O(n)               |
| Task 1 | display_communities() | BST            | O(1)       | O(n)       | O(n)               |
| Task 1 | input_supplier()      | Array          | O(1)       | O(n)       | O(n)               |
| Task 1 | input_warehouse()     | Array          | O(1)       | O(n)       | O(n)               |
| Task 1 | input_community()     | BST + Array    | O(log n)   | O(n log n) | O(n <sup>2</sup> ) |
| Task 2 | heapify_max()         | Max Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | build_max_heap()      | Max Heap       | O(n)       | O(n)       | O(n)               |
| Task 2 | insert_area()         | Max Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | removeTopArea()       | Max Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | heapify_min()         | Min Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | build_min_heap()      | Min Heap       | O(n)       | O(n)       | O(n)               |
| Task 2 | insert_food()         | Min Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | RemoveTopFood()       | Min Heap       | O(1)       | O(log n)   | O(log n)           |
| Task 2 | display_areas()       | Max Heap       | O(n log n) | O(n log n) | O(n log n)         |
| Task 2 | display_food()        | Min Heap       | O(n log n) | O(n log n) | O(n log n)         |
| Task 2 | Output Loops (all 3)  | Both Heaps     | O(n log n) | O(n log n) | O(n log n)         |

## Task 3 :

### Cost Effective Distribution Model (Requirement 5)

#### Graph:

A **Graph** is a collection of points called nodes connected by lines called edges. Each edge can have a number on it called a weight. In our case, the weight is the fuel cost to travel between two areas.

In our food redistribution system:

Node = a distribution area in Karachi

Edge = a road connecting two areas

Weight = fuel cost to travel that road



Date: \_\_\_\_\_

## 15 Distribution Areas

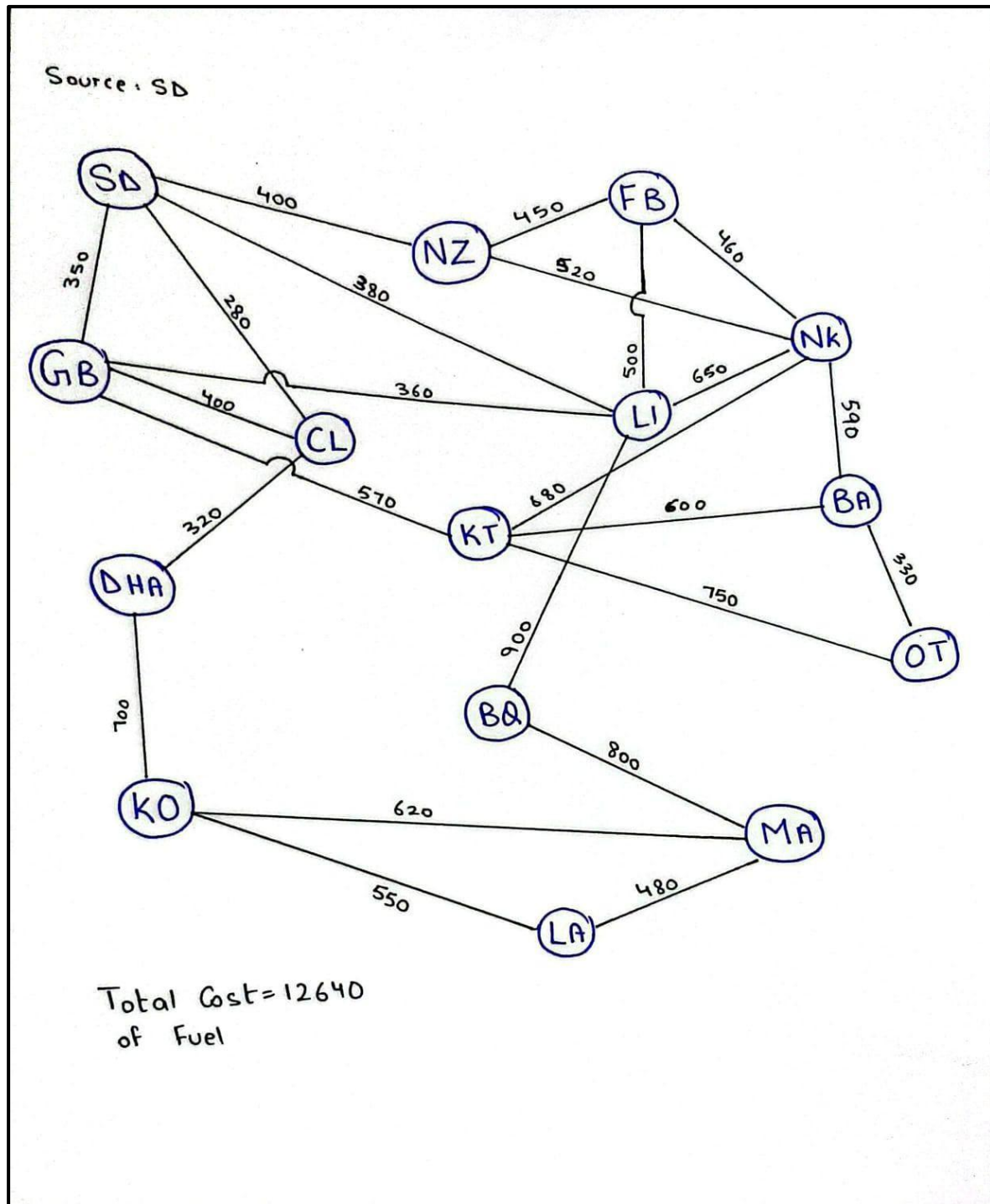
- 1- Saddar => SD
- 2- Nazimabad => NZ
- 3- Clifton => CL
- 4- Gulberg => GB
- 5- Karachi Town => KT
- 6- Liaquatabad => LI
- 7- Bin Qasim => BQ
- 8- Defence => DHA
- 9- Korangi => KO
- 10- Landhi => LA
- 11- Orangi Town => OT
- 12- Baldia => BA
- 13- North Karachi => NK
- 14- Malir => MA
- 15- Federal B Area => FB

Date: \_\_\_\_\_

## Adjacency List:

|     |   |                        |
|-----|---|------------------------|
| SD  | → | GB , CL , LI , NZ      |
| NZ  | → | SD , FB , NK           |
| CL  | → | SD , GB , DHA          |
| GB  | → | SD , LI , CL , KT      |
| KT  | → | GB , NK , BA , OT      |
| BQ  | → | LI , MA                |
| DHA | → | CL , KO                |
| KO  | → | MA , LA , DHA          |
| LA  | → | KO , MA                |
| OT  | → | KT , BA                |
| NK  | → | FB , NZ , LI , KT , BA |
| BA  | → | KT , OT , NK           |
| LI  | → | GB , SD , FB , NK , BQ |
| MA  | → | LA , KO , BQ           |
| FB  | → | NZ , LI , NK           |

## NETWORK GRAPH REPRESENTATION





| Prims Algorithm Table      |       |       |                |
|----------------------------|-------|-------|----------------|
| Starting from SD (saddor). |       |       |                |
| V                          | known | dv    | P <sub>v</sub> |
| SD                         | 0 1   | ∞ 0   | 0              |
| NZ                         | 0 1   | ∞ 400 | ∅ SD           |
| CL                         | 0 1   | ∞ 280 | ∅ SD           |
| GB                         | 0 1   | ∞ 350 | ∅ SD           |
| KT                         | 0 1   | ∞ 570 | ∅ GB           |
| LI                         | 0 1   | ∞ 360 | ∅ GB           |
| BQ                         | 0 1   | ∞ 800 | ∅ MA           |
| DHA                        | 0 1   | ∞ 320 | ∅ CL           |
| KO                         | 0 1   | ∞ 700 | ∅ DHA          |
| LA                         | 0 1   | ∞ 550 | ∅ KO           |
| OT                         | 0 1   | ∞ 330 | ∅ BA           |
| BA                         | 0 1   | ∞ 590 | ∅ NK           |
| NK                         | 0 1   | ∞ 460 | ∅ FB           |
| MA                         | 0 1   | ∞ 480 | ∅ LA           |
| FB                         | 0 1   | ∞ 450 | ∅ NZ           |

### PRIM'S MST WORKING

1. Start from one area (Saddar)
2. Mark it as visited
3. Look at all roads going from visited areas to unvisited areas
4. Pick the road with the LOWEST fuel cost
5. Add that area to visited
6. Repeat steps 3-5 until all 15 areas are visited

**Prim's Algorithm: Step by step calculation for above Table**

Step 1: For SD

$$SD \rightarrow NZ = 400$$

$$SD \rightarrow CL = 280 \leftarrow \text{Cheapest}$$

$$SD \rightarrow GB = 350$$

$$SD \rightarrow LI = 380$$

Step 2: For CL

$$CL \rightarrow SD = \text{Already known}$$

$$CL \rightarrow GB = 400$$

$$CL \rightarrow DHA = 320$$

Step 3: For DHA

$$DHA \rightarrow CL = \text{Already known}$$

$$DHA \rightarrow KO = 700$$

Step 4: For GB

$$GB \rightarrow SD = \text{Already known}$$

$$GB \rightarrow CL = \text{Already known}$$

$$GB \rightarrow LI = 360 \quad (360 < 380) \text{ update LI}$$

$$GB \rightarrow KT = 510$$

Step 5: For LI

$$LI \rightarrow SD = \text{Already known}$$

$$LI \rightarrow GB = \text{Already known}$$

Date: \_\_\_\_\_

$$LI \rightarrow FB = 500$$

$$LI \rightarrow NK = 650$$

$$LI \rightarrow BA = 900$$

Step 6 : NZ

$$NZ \rightarrow SD = \text{known}$$

$$NZ \rightarrow FB = 450 \quad (450 < 500) \text{ update FB}$$

$$NZ = NK = 520 \quad (520 < 650) \text{ update NK}$$

Step 7: For FB

$$FB \rightarrow NZ = \text{known}$$

$$FB \rightarrow LI = \text{known}$$

$$FB \rightarrow NK = 460 \quad (460 < 520) \text{ update NK}$$

Step 8 : For NK

$$NK \rightarrow FB = \text{known}$$

$$NK \rightarrow NZ = \text{known}$$

$$NK \rightarrow LI = \text{known}$$

$$NK \rightarrow KT = 680$$

$$NK \rightarrow BA = 590$$

Step 9 : For KT

$$KT \rightarrow GB = \text{known}$$

$$KT \rightarrow NK = \text{known}$$

$$KT \rightarrow BA = 600$$



Date: \_\_\_\_\_

$$KT \rightarrow OT = 750$$

Step 10 : BA

$$BA \rightarrow KT = \text{Known}$$

$$BA \rightarrow NK = \text{Known}$$

$$BA \rightarrow OT = 330 (330 < 750) \text{ update OT}$$

Step 11 : OT

$$OT \rightarrow BA = \text{Known}$$

$$OT \rightarrow KT = \text{Known}$$

Step 12 : KO

$$KO \rightarrow DHA = \text{Known}$$

$$KO \rightarrow MA = 620$$

$$KO \rightarrow LA = 550$$

Step 13 : LA

$$LA \rightarrow MA = 480 (480 < 620) \text{ update MA}$$

$$LA \rightarrow KO = \text{Known}$$

Step 14 : MA

$$MA \rightarrow BA = 800 (800 < 900) \text{ update BA}$$

$$MA \rightarrow KO = \text{Known}$$

$MA \rightarrow LA = \text{known}$

Step 15: BQ

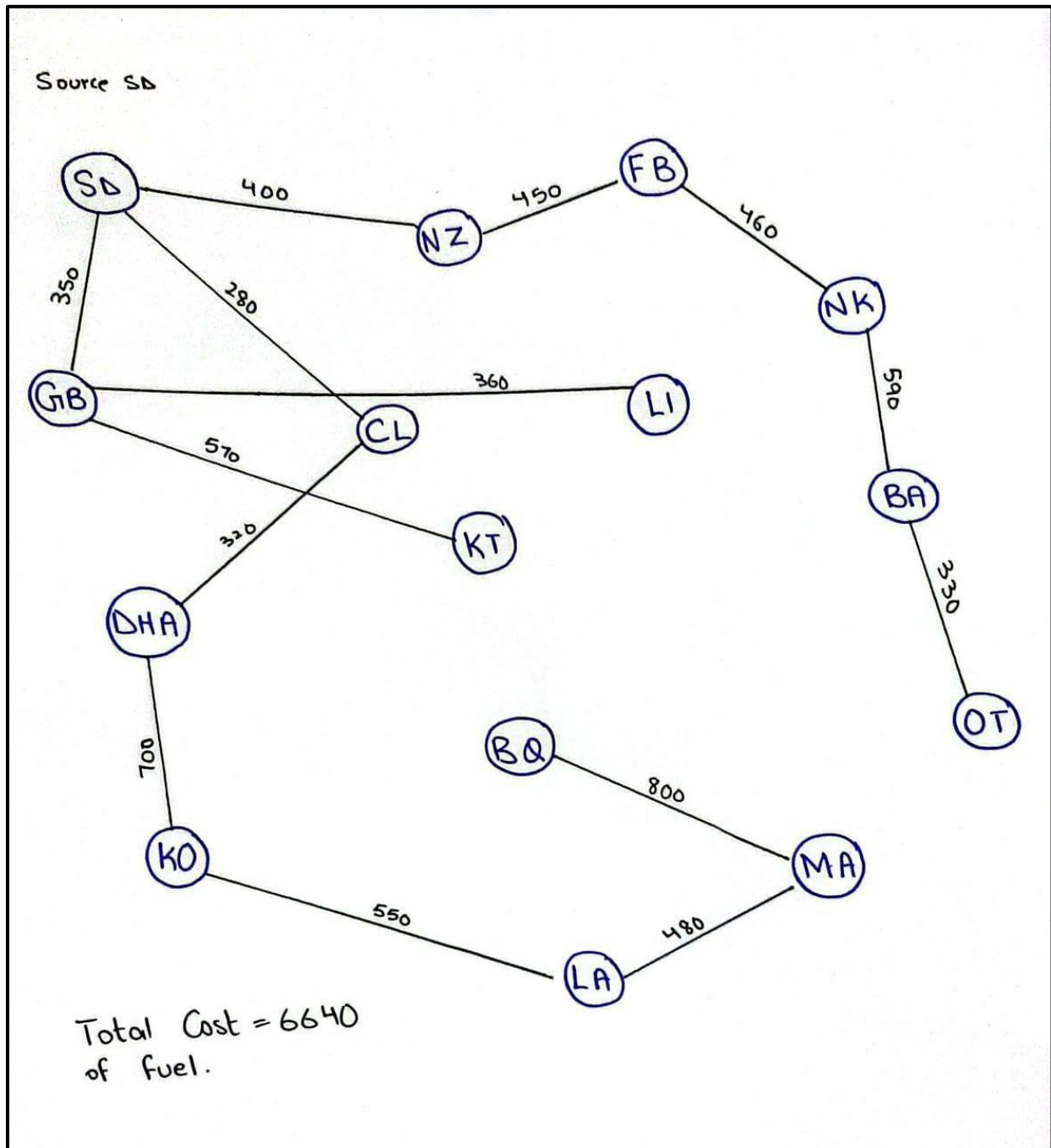
$BQ \rightarrow LI = \text{known}$

$BQ \rightarrow MA = \text{known}$



### MST Network Graph.

Starting point ->SADDAR (SD)



**Comparison: Normal Network vs MST Network**

| <b>Features</b>      | <b>Normal Network</b> | <b>MST NETWORK</b> |
|----------------------|-----------------------|--------------------|
| Total roads used     | 24 Roads              | 14 roads           |
| Total Fuel Cost      | Rs. 12,640            | Rs. 6,640          |
| Fuel saved           | 0                     | Rs. 6,000          |
| Saving percentage    | 0                     | 47.47%             |
| Redundant road       | Yes                   | No                 |
| Cycle/Loops          | Yes                   | No                 |
| All 15 areas covered | Yes                   | Yes                |
| Algorithm used       | None                  | Prim's MST         |

The MST network saves Rs. 6,000 in fuel 47.47% reduction while still delivering food to all 15 areas of Karachi without missing a single one.

# Explanation and Justification of Data Structures

## 1. struct : For Suppliers, Warehouses, Communities, Food Items

### What it is:

A struct groups multiple related variables into one named unit. Instead of having separate arrays for names, populations, food requirements everything about one entity lives together.

```
struct community {
 string area_name;
 int population;
 int food_required;
 int poverty_level;
};
// Now one variable holds all info about one area
community c1;
c1.area_name = "Orangi";
c1.population = 800000;
c1.food_required = 2000;
c1.poverty_level = 6;
```

### Why chosen here:

- Suppliers have name, type, food available which belong together
- Warehouses have name, location, capacity, stock which belong together
- Communities have name, population, food required, poverty level which belong together
- Using struct makes code readable and organized

### Efficiency:

- $O(1)$  to access any field instant
- no searching needed ,no traversal needed.
- suppliers[0].food\_available gives the answer instantly.

### Scalability:

- Adding more fields (e.g. GPS coordinates, contact number) is easy, just add to the struct
- Works for any number of suppliers or warehouses

### Suitability:

- Every entity in this system is a real world object:

Supplier = has a name, type, and food amount

Warehouse = has a name, location, capacity, stock

Community = has a name, population, food need, poverty level

Food item = has a name and expiry days

- struct is the most natural and direct way to model

real world objects

## 2. BST (Binary Search Tree) For Communities (Task 1)

### What it is:

A tree where every node follows one rule: left child has a smaller value, right child has a larger value. In our system, communities are inserted and sorted by `poverty_level`

// From Task 1

```
struct BST_Node {
 community data;

 BST_Node* left;

 BST_Node* right;

};
```

Communities inserted in order: 6, 5, 7, 8, 4

Orangi(6), Malir(5), Nazimabad(7), Korangi(4), Lyari(8)

In-order traversal gives: 4, 5, 6, 7, 8

= Korangi, Malir, Orangi, Nazimabad, Lyari

automatically sorted lowest to highest poverty

### Efficiency:

- Insert:  $O(\log n)$  go left or right at each level, half tree eliminated
- Search:  $O(\log n)$  same logic

- Much better than array where insert requires shifting =  $O(n)$

### Scalability:

- Works well as more areas are added over time
- 1,000 areas needs only about 10 comparisons to find any area

### Suitability:

- Natural fit for this problem because areas need to be sorted by poverty level
- Poverty level is the key BST organizes by key automatically
- Search by poverty level is a direct BST operation

### Why Not Other Structures?

Plain array  $\rightarrow O(n)$  search,  $O(n)$  insert (shifting)  $\rightarrow$  too slow

Linked list  $\rightarrow O(n)$  search, no ordering  $\rightarrow$  not suitable

Hash table  $\rightarrow O(1)$  search but no sorted order and cannot do in-order traversal

Heap  $\rightarrow O(n)$  search  $\rightarrow$  violates  $O(\log n)$  requirement

BST  $\rightarrow O(\log n)$  average, better to implement  $\rightarrow$  best fit for this level

## 3. Max-Heap (Task 2, Requirement 3)

### What it is:

A Max Heap is an array based tree where the largest value always sits at index 0. Every parent is always greater than or equal to its children. In our system, the area with the highest poverty\_level is always at the top.

### Why chosen here:

- Requirement 3 says: serve the area with highest poverty score first
- Max-Heap guarantees the highest score is always at index 0 —  $O(1)$  to access
- After removing top, heap fixes itself automatically in  $O(\log n)$
- Far better than sorting array every time someone is served — that would be  $O(n \log n)$  each time

### Efficiency:

| Operation     | How it works                                  | Time        |
|---------------|-----------------------------------------------|-------------|
| insert_area() | Place at end, bubble up comparing with parent | $O(\log n)$ |

|                 |                                           |             |
|-----------------|-------------------------------------------|-------------|
| removeTopArea() | Remove index 0, fix with heapify_max      | $O(\log n)$ |
| heapify_max()   | Push element down comparing with children | $O(\log n)$ |

**Scalability:**

- Adding 1,000 areas still means only 10 steps for insert and remove
- The system can handle Karachi's full 20+ million population across hundreds of areas without slowing down.

**Suitability:**

Requirement 3 says:

"Provide the food first where poverty is higher."

Max-Heap is the PERFECT data structure for this because:

- Highest poverty\_level is ALWAYS at index 0
- After each delivery (removeTopArea), the next
- highest area automatically rises to the top
- No sorting step needed heap maintains order automatically on every insert and remove
- $O(\log n)$  per operation is efficient and fast

Always serve the highest priority first.

**Why not other data structures?**

- Plain array (re-sort each time)  $\rightarrow O(n \log n)$  per delivery  $\rightarrow$  too slow
- Linked list (sorted insertion)  $\rightarrow O(n)$  per insert  $\rightarrow$  too slow
- BST  $\rightarrow O(\log n)$  but complex for priority  $\rightarrow$  not ideal
- Max Heap  $\rightarrow O(\log n)$  guaranteed, top always ready  $\rightarrow$  perfect fit

**4. Min-Heap For Perishable Food Priority (Task 2, Requirement 4)****What it is:**

A Min-Heap is the inverse of Max-Heap. The smallest value always sits at index 0. Every parent is always less than or equal to its children. In our system, the food item with the fewest days left (expires soonest) is always at the top.

**Why chosen here:**

- Requirement 4 says distribute food expiring soonest first to prevent waste
- Min Heap guarantees the food with fewest days left is always at index 0
- After distributing it, heap fixes itself and next soonest expiry item rises to top

**Efficiency:**

| Operation       | How it works                               | Time        |
|-----------------|--------------------------------------------|-------------|
| insert_food()   | Place at end, bubble up comparing daysleft | $O(\log n)$ |
| RemoveTopFood() | Remove index 0, fix with heapify_min       | $O(\log n)$ |
| heapify_min()   | Push element down comparing with children  | $O(\log n)$ |

### Scalability:

- Handles hundreds of food items without slowing down
- Each new food item inserted in  $O(\log n)$  regardless of total count

### Suitability:

Requirement 4 says:

"Distribute the food perishable items first."

Min-Heap is the PERFECT data structure for this because:

- Food with fewest daysleft is ALWAYS at index 0
- After distributing that food (RemoveTopFood), the next soonest expiry food automatically rises to top
- This directly prevents food waste no item ever sits waiting while something else expires
- $O(\log n)$  insert and remove efficient for real time use
- Through Min Heap models there is no need for manual management.

### Why not other structures?

**Plain array** (sort by expiry)  $\rightarrow O(n \log n)$  per distribution  $\rightarrow$  too slow

**Linked list** (sorted by expiry)  $\rightarrow O(n)$  per insert  $\rightarrow$  too slow

**BST**  $\rightarrow O(\log n)$  but extra complexity (task 2 does not need search and sorted display)  $\rightarrow$  not needed

**Hash Table**  $\rightarrow$  has no concept of priority or ordering. It cannot tell you which area has the highest poverty level or which food expires soonest without scanning everything.

**Sorting Algorithms** Same problem as Task 1 but worse. After every single delivery, the array would need to be re sorted to find the next highest priority area. That is  $O(n \log n)$  per delivery. Heap maintains priority automatically after every removal in  $O(\log n)$ .

**Min-Heap**  $\rightarrow O(\log n)$  guaranteed, min always ready  $\rightarrow$  perfect fit

## 5. Graph For Distribution Routes (Task 3, Requirement 5)

### What it is:

A graph consists of nodes (areas) connected by edges (roads). Each edge has a weight (fuel cost). Our graph is **undirected** because roads go both ways with the same fuel cost, and **weighted** because each road has a specific fuel cost in Rupees.

### Why chosen here:

- A city road network is naturally a graph every area is a node, every road is an edge
- Fuel cost is the weight of each edge
- Adjacency matrix gives  $O(1)$  to check if any two areas are directly connected

### Efficiency:

- $O(1)$  to check road between any two areas

Adjacency List:

Space:  $O(V + E)$  where  $V$  = areas,  $E$  = roads

Check if road exists:  $O(\text{degree of node})$

Good for sparse graphs (fewer roads than  $V^2$ )

For our 15 areas with 24 roads:

$V = 15, E = 24 \rightarrow$  much less than  $V^2 = 225$

Adjacency list is efficient for this density

### Scalability:

As more areas of Karachi are added:

Simply add more nodes to the list

Add edges between connected areas

Prim's algorithm scales as  $O(V^2)$

For 100 areas  $\rightarrow$  10,000 operations  $\rightarrow$  still fast

### Suitability:



Requirement 5 says:

"Model the distribution route."

A graph is the ONLY natural way to model a road network:

- > Every area = one node
- > Every road between two areas = one edge
- > Every fuel cost = edge weight
- > The entire Karachi delivery network = one connected graph

No other data structure can represent "which areas connect to which other areas with what cost" as naturally and efficiently as a graph.

## 6. Prim's MST Algorithm For Minimum Fuel Routes (Task 3)

### What it is:

An algorithm that finds the minimum cost set of roads to connect all areas — no loops, no wasted routes, every area reachable.

### Why chosen here:

- Requirement 5 asks to minimize fuel cost
- MST connects all 15 areas with minimum total fuel exactly what is needed
- No cycles means no truck ever takes a redundant route

### Efficiency:

With  $V = 15$  areas:

Prim's with adjacency list =  $O(V^2) = 15 \times 15 = 225$  operations

This is extremely fast runs in microseconds.

### Scalability:

- Works for any size network
- For 100 areas:  $100 \times 100 = 10,000$  operations, still fast

### Suitability:

- Directly solves "minimize fuel cost" MST is the standard algorithm for this exact problem
- Guarantees every area is reachable at minimum total cost
- $V = 50$  areas -> 2,500 operations
- $V = 100$  areas -> 10,000 operations

- Still very fast for practical city scale networks.

**Requirement 5 says:**

"Cost of fuel is minimized."

Prim's MST is the DIRECT solution to this requirement:

- Connects ALL 15 areas (no area missed)
- Uses MINIMUM total fuel cost
- No cycles = no wasted/redundant routes
- Exactly  $n-1 = 14$  roads for 15 areas

Normal Network: Rs. 12,640 (all 24 roads)

MST Network: Rs. 6,640 (14 optimal roads)

Fuel Saved: Rs. 6,000 (47.47% reduction)

This directly translates to more food delivered per Rupee of fuel. Perfectly aligned with the goal of helping underprivileged communities efficiently.

## Overall Data Structure Choice Summary

| Data Structure      | Used For                                | Why Best Choice                                                 |
|---------------------|-----------------------------------------|-----------------------------------------------------------------|
| struct              | All entities                            | Groups related data, $O(1)$ access, clean and readable          |
| BST                 | Storing communities sorted by poverty   | $O(\log n)$ insert and search, auto sorted                      |
| Max Heap            | Serving highest poverty area first      | $O(\log n)$ , guaranteed highest always at top                  |
| Min Heap            | Distributing most perishable food first | $O(\log n)$ , guaranteed lowest always at top                   |
| Graph (Adj. Matrix) | Modeling road network between 15 areas  | Only structure that naturally represents connections with costs |
| Prim's MST          | Finding minimum fuel cost routes        | $O(V^2)$ , guarantees minimum cost connection of all areas      |